

前言

通过前四章的学习，你已熟悉如何搭建 类库模块 与 多包仓库 的前端基建。通过 前端工程化 的手段打造了属于自己的轮子，在编码方面已完成很多需求，然而一个良好实用的前端基建系统还需完善一些看似不是很重要的细节。

这些细节有可能因为入口文件的优先级别导致构建流程出错，有可能因为软件的版权问题导致使用源码时受到影响，有可能因为文档的排版布局不够出色不能引人注目等问题。

前端基建系统最终还是会开源到公司内网或外网，所以在发布前必须将这些细节完善，确保开源项目不受这些小问题的影响。本章将带领你**完善开源输出的发布准备**，从 项目配置文件、软件许可证书 和 项目自述文档 三方面入手，确保源码在上线前拥有基础保障，使其他开发者 看得省心用得放心。

方案：优化项目配置文件

Package指项目配置文件，用于定义整个项目所需的配置信息，包括但不限于描述配置、文件配置、脚本配置、依赖信息、发布配置、第三方配置等字段。**Package**对应实体文件是 `package.json`，没错，就是经常 `npm i` 时引用到的文件。

很多同学对 `package.json` 配置字段的印象是又多又乱。为了方便对比，通过下图展示它们各自的组成与作用，相信你有更好的理解。

package.json

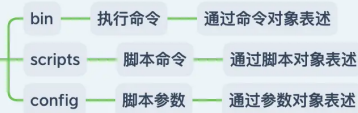
描述配置



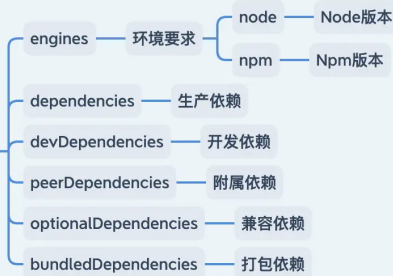
文件配置



脚本配置



依赖配置



发配置



其中有几个配置字段尤为重要，以第15章封装的工具库为例，以下将一一展开讲述。

版本编号

`version` 表示版本编号，用于确定项目在每个阶段的唯一性。在项目每次定版发布前，需更新一次版本。执行 `npm view react versions`，输出以下信息，可能都会经常见到这些版本，但很少同学会主动了解这些版本为何这样设计。

```
"16.0.0-alpha",
"16.0.0-alpha.1",
"16.0.0-alpha.2",
"16.0.0-alpha.3",
...
"16.0.0-beta",
"16.0.0-beta.1",
"16.0.0-beta.2",
"16.0.0-beta.3",
...
"16.0.0-rc",
"16.0.0-rc.1",
"16.0.0-rc.2",
"16.0.0-rc.3",
...
"16.0.0",
"16.1.0",
"16.1.1",
"16.2.0",
...
```

从 `react` 的版本列表不难得出以下结论。

- 版本的形式严格遵循 `a.b.c`
- 版本的迭代严格遵循 右位递增
- 发布重要版本前，可发布 `alpha`、`beta`、`rc` 等先行版本
- 先行版本的编号后方可带上 次数 或 元信息

在早期的软件发展道路中，安装某个软件包时会发现该软件包中又依赖不同指定版本的其它软件包。随着系统功能越来越复杂，依赖软件包越来越多，其依赖关系也越来越深，可能面临版本控制被锁死的风险，这就是**依赖地狱**。

基于该原因，Github 起草了一份具备指导意义且统一版本编号的表示规则，称为 语义化版本 (Semantic Versioning)，简称 Semver 。目前它由 Npm团队 维护。

Semver 的出现规定了版本如何表示如何迭代如何比较。遵从 Semver规范 的 Npm模块 ，其依赖会很清晰，不会出现循环依赖、依赖冲突等常见问题。关于 Semver 的完整规范可查看[Semver官网](#)。

版本的命名完全遵循 Semver规范 ，格式为 主版本.次版本.修订版本 。

版本号	简称	又称	功能	描述
主版本	major	Breaking Change	在原来功能框架中新增功能	通常是 API 自
次版本	minor	New Feature	在原来功能框架中新增功能	通常是功能改
修订版本	patch	Bug Fix	在原来功能框架中修复缺陷	通常是缺陷改



先行版本的 次数 与 元信息 可加到 主版本.次版本.修订版本 的后方作为延伸。

先行版本

为何要使用先行版本？若某个版本改动较大且不稳定，可能无法满足预期的兼容性需求，就需发布先行版本作为铺垫或过渡，所以很多明星项目都会这样处理。先行

版本通过 - 连接编号与 次数 或 元信息 。

标记	简称	功能	描述
内测版本	alpha	主要以实现软件功能为主	只在开发者内部交流，问题较多需继续优化
公测版本	beta	主要以修复问题缺陷为主 还会加入新功能	可在社区外部交流，问题不多但需继续优化
候选版本	gama 或 rc	主要以消除最终错误为主 不会加入新功能	与正式版本无异

当经过先行版本一系列测试后，终归会有一个正式版本，该版本是最终交付到用户使用的一个版本，也就是 `release` 。例如 `react` 的[releases](#)。

版本准则

标准的版本必须严格遵循 `a.b.c` 且三者都为非负整数，禁止在数字前方补零，版本发布需严格递增。例如 `1.1.1 -> 1.2.0 -> 1.2.1` 。版本发行后，任何改动都必须以新版本发行。

`1.0.0` 的版本用于界定公共 `API` 。当发布软件到正式环境或存在稳定 `API` 时就可发布 `1.0.0` 的版本了，因此 `1.0.0` 不是随便定义的。

版本的优先层级指不同版本在排序时如何比较。判断优先层级时，必须把版本依序拆分为 `主版本` 、 `次版本` 、 `修订版本` 和 `先行版本` 后比较。

版本规则

在执行 `npm i react` 时会在 `package.json` 中生成以下描述。当执行安装命令安装 `Npm` 模块 时， `Npm` 会首先安装依赖的最新版本，然后将包名及版本编号写入到 `package.json` 中。被安装依赖的版本前方会默认加上 `^` 。

json

```
{
  "dependencies": {
    "react": "^18.0.0"
  }
}
```

除了 `^`，还有 `~`、`<`、`>`、`=`、`<=`、`>=`、`-`、`||`、`x`、`*`。

`^` 表示同一 **主版本** 中不小于指定版本的版本。 `^2.2.1` 对应主版本为 **2**，不小于 **2.2.1** 的版本，例如 **2.2.1**、**2.2.2**、**2.3.0** 等，**主版本** 固定。

`~` 表示同一 **主版本** 与 **次版本** 中不小于指定版本的版本。 `~2.2.1` 对应主版本为 **2**，次版本为 **2**，不小于 **2.2.1** 的版本，例如 **2.2.1**、**2.2.2** 等，**主版本** 与 **次版本** 固定。

`>`、`<`、`=`、`>=`、`<=`、`-` 表示一个版本范围。 `-` 必须使用前后空格间隔。

json

```
{
  "engines": {
    "node": "16.0.0 - 16.14.0",
    "npm": ">=7.10.0"
  }
}
```

`||` 表示满足多个条件的版本。 `||` 必须使用前后空格间隔。

json

```
{
  "engines": {
    "node": ">=16.0.0 || <=16.14.0",
    "npm": ">=7.10.0"
  }
}
```

`x` 与 `*` 表示通配版本。

json

```
{
  "engines": {
    "node": "16.x",
    "npm": "*"
  }
}
```

通常发布一个 Npm 模块到 Npm 公有仓库，正确做法是先修改 `package.json` 的 `version`，然后执行 `npm publish` 发布模块。手动修改版本的做法建立在你特别熟悉 `Semver` 规范的基础上，否则可能会造成版本混乱。Npm 考虑到这点就提供相关命令让开发者更好地遵循 `Semver` 规范。

- 升级主版本： `npm version <major>`
- 升级次版本： `npm version <minor>`
- 升级修订版本： `npm version <patch>`

入口文件

`main/module/browser` 同时表示入口文件，到底它们存在何种关系，又存在优先级别吗？在此引申出一个新问题，`require` 与 `import` 引用的 Npm 模块，到底是使用哪个字段加载入口文件？

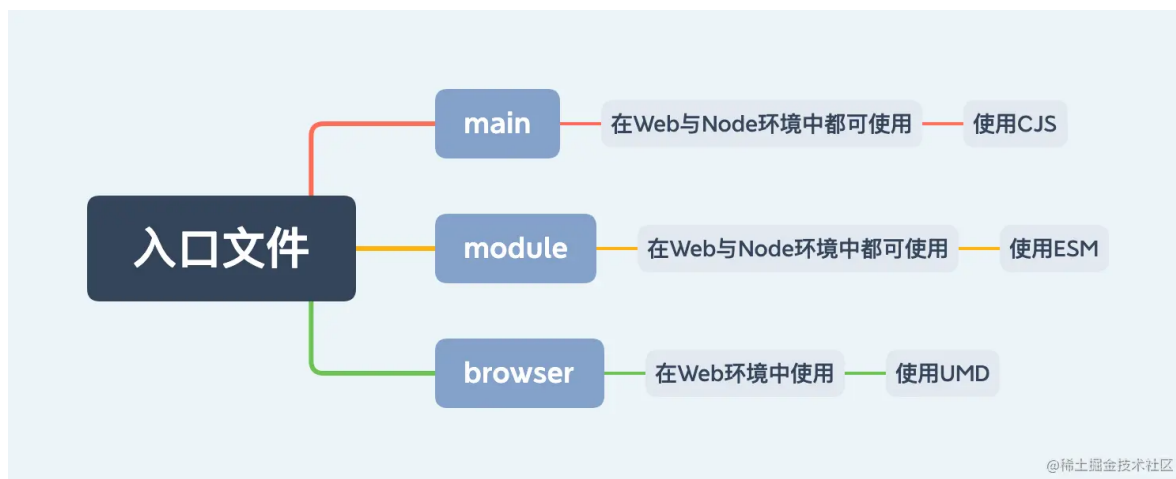
第15章开发工具库时就埋下该问题。在开发前就明确清楚 Npm 模块的应用场景。

- 只能在 Web 中使用
- 只能在 Node 中使用
- 可在 Web 与 Node 中使用

这貌似对应了三种主流的模块规范，分别是 ESM、CJS 和 UMD。ESM 容易静态分析，可启用摇树优化删除所有未实际使用的代码，推荐首选；CJS 使用同步加载适用于 Node 环境，无摇树优化；UMD 是 CJS 与 ESM 的结合体，可自动挂在全局导出 `global`，适用于 Web 但也兼容 Node。

第15章开发的工具库可在 Web 与 Node 中使用，那仅仅一个 `main` 字段是无法保障在 Web 中加载哪个文件，在 Node 中又加载哪个文件，因此在不同环境中加载 Npm 模块不同入口文件，显然一个 `main` 字段已不能够满足需求，这就衍生出 `module` 与 `browser` 两个字段。

根据 CJS/ESM/UMD 的特性与 Npm 官网对 `main/module/browser` 字段的描述，可得到以下对应关系。



main

`main` 字段在 Web 与 Node 环境中都可用。若将项目发布为 Npm 模块，使用 `require` 或 `import` 导入它时，默认返回的入口文件就是 `main` 字段指向入口文件的 `module.exports` 或 `export/export default`。若不指定该字段，默认返回是根目录中的 `index.js`，若入口文件未找到则直接报错。

module

`main` 字段在 Web 与 Node 环境中都可用，前提是入口文件必须使用 ESM。更多时候是 `main` 与 `module` 混用，使用 CJS 书写是为了用户在配置 `babel` 时可放心屏蔽 `node_modules` 文件夹，使用 ESM 书写是为了用户在使用 Npm 模块时可享受摇树优化带来的好处。

```
{  
  "main": "dist/web.js",  
  "module": "dist/web.esm.js"  
}
```

json

这样相当于一个 Npm 模块内同时发布两种 模块规范 的版本。当打包工具遇到这种 Npm 模块时，若能识别 `module` 字段则会优先使用 ESM 版本的入口文件，这样可启用摇树优化机制，若不能识别 `module` 字段则会使用 CJS 版本的入口文件，这样也不会阻碍打包流程。

在 `module` 字段未作为 `package.json` 标准字段前，社区默契地形成使用 `jsnext:main` 字段代表 `module` 字段，因此引用一些老旧但下载量很高的 Npm 模块，还是能看到该字段的存在，而 `webpack` 也一直能识别该字段。


```
{  
  "main": "dist/web.js",  
  "jsnext:main": "dist/web.esm.js"  
}
```

browser

`browser` 字段只能在 Web 环境中可用。若 Npm 模块 只在 Web 中使用且严禁在 Node 中使用，那使用 `browser` 指定入口文件再也适合不过了。

综上所述，为了能让打包工具更快更准地寻找入口文件，Npm 模块 就必须根据以下情况配置相关字段。

- 若 Npm 模块 只导出 CJS 的入口文件，使用 `main` 字段
- 若 Npm 模块 只导出 ESM 的入口文件，使用 `module` 字段
- 若 Npm 模块 导出 CJS 与 ESM 的入口文件，使用 `main/module` 字段
- 若 Npm 模块 只在 Web 中使用且严禁在 Node 中使用，使用 `browser` 字段
- 若 Npm 模块 只在 Node 中使用，使用 `main` 字段
- 若 Npm 模块 能在 Web 与 Node 中使用，使用 `main/browser` 字段
- 若 Npm 模块 导出 CJS 与 ESM 的入口文件且能在 Web 与 Node 中使用，使用 `main/module/browser` 字段

针对第15章开发的工具库，很明显符合最后一种情况，在 `package.json` 中指定以下字段。

```
{  
  "main": "dist/web.js",  
  "jsnext:main": "dist/web.esm.js",  
  "module": "dist/web.esm.js",  
  "browser": "dist/web.umd.js"  
}
```

为了让各个版本 `webpack` 都能正确识别上述字段，根据字段权重配置 `resolve-mainFields`。`webpack` 在执行时，遇到 Npm 模块 会优先解析 `jsnext:main`，然后解析 `module`，然后解析 `browser`，最后解析 `main`。

```
export default {  
  resolve: {  
    mainFields: [  
      "jsnext:main", // ESM  
      "module", // ESM  
      "browser", // UMD  
      "main" // CJS  
    ] // 导入模块入口描述  
  }  
};
```

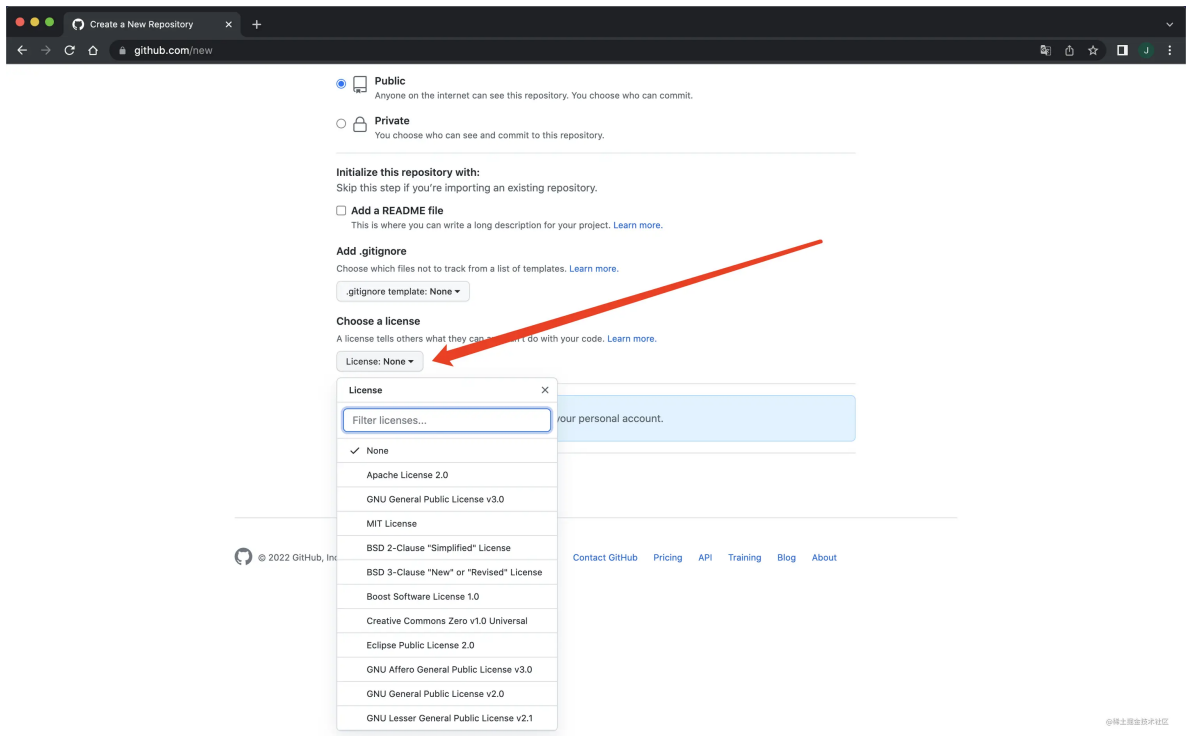
方案：优化软件许可证书

License指软件许可证书，又称 **开源协议**，用于授权或约定使用者可拥有的权利与需遵从的义务。**License** 对应实体文件是 **license** 或 **LICENSE**。

现在很多优秀的开源项目都有配置开源协议，不同开源协议的约束条件也不同，因此开源不等于免费，开源也不等于无约束。乱用开源协议但不遵守法律，那就真的可能会收到律师函了。

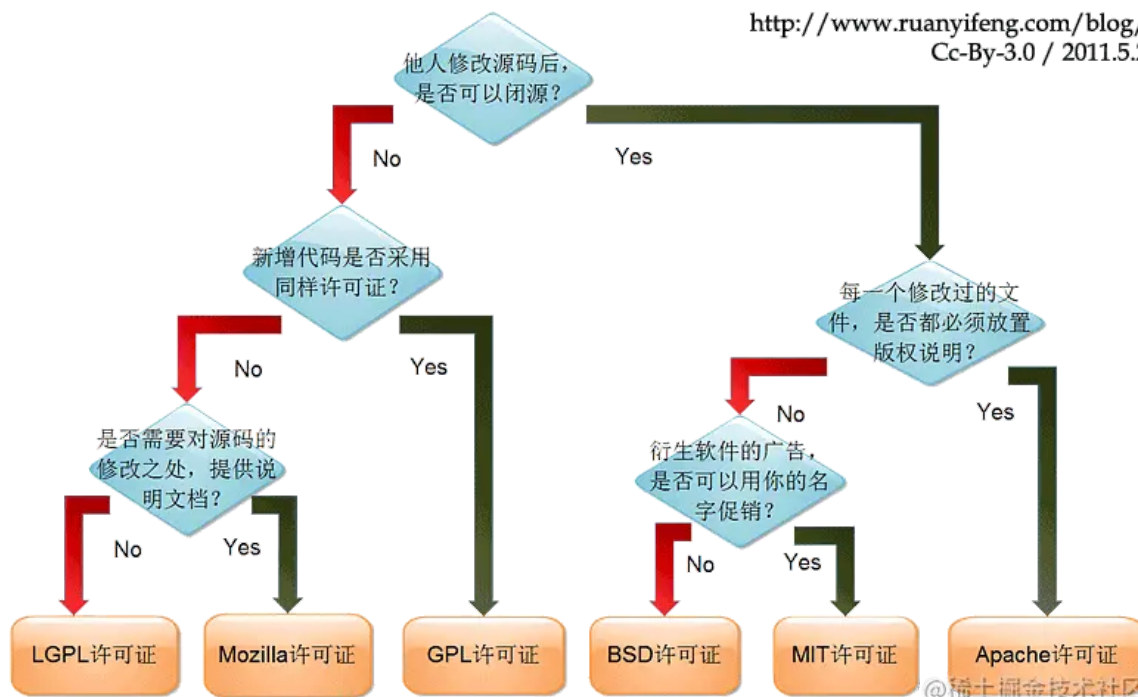
对于大型项目，可能有专业的律师团队撰写开源协议。可是作为一名开发者，有时想开源自己的项目但又不想自己的源码被随意借鉴、分享和抄袭，就必须撰写开源协议约束一些被行业反感的行爲。

开源协议往往需具备专业知识，况且它涉及法律规则，普通人不可能在短时间内就掌握这些知识。**Github** 就提供了一系列流行的开源协议满足项目需求，打开[创建仓库](#)，点击 **License:None**，任君选择。



协议	简述
AGPL	拓展 GPL3 ，使用在线网络服务也需开源
Apache	允许他人修改源码后闭源，需对每个被改动的文件做版权说明
BSD2/BSD3	与 MIT 相似，未经事先书面许可不得使用版权所有者信息做推广
BSL	与 GPL3 相似，无需复制版权说明
CCZ	放弃作品版权并将其奉献给大众，不对源码做任何担保
EPL	与 GPL3 相似，有权使用、修改、复制和发布软件原始版本与迭代版本，但在某些情况下需将改动内容释出
GPL2	与 GPL3 相比，若使用源码作为服务提供而不分发软件则无需开源
GPL3	无论以何种方式修改或使用源码，需开源
LGPL	与 GPL3 相比，允许商业软件通过类库引用使用类库而无需开源
MIT	允许他人修改源码后闭源，无需对每个被改动的文件做版权说明，二次开发可用原作者信息做推广
Mozilla	与 LGPL 相似，需对每个被改动的文件做版权说明
Unlicense	与 CCZ 相似，允许开放商标与专利授权

若还有不懂可查看阮一峰老师对开源协议的图例。



当然不选择开源协议也行，若将源码托管到 [Github公有仓库](#) 中，会默认公开源码且所有开发者都能自由 [View](#) 或 [Fork](#) 。当然还是在项目中撰写一份开源协议约束一些被行业反感的行为会更好。

- [MIT](#) 是一份很宽松的开源协议，它允许对源码做任何形式的改动与推广，若不知如何选择就使用该协议
- [GPL](#) 是一份很自由的开源协议，它鼓励免费，若不介意的话可用该协议
- [BSD](#) 是一份很严格的开源协议，它未经事先书面许可不得使用，若介意的话可用该协议

在此贴下[MIT](#)的协议模板，毕竟它是最多项目使用的协议无之一，复制粘贴修改主体信息即可。

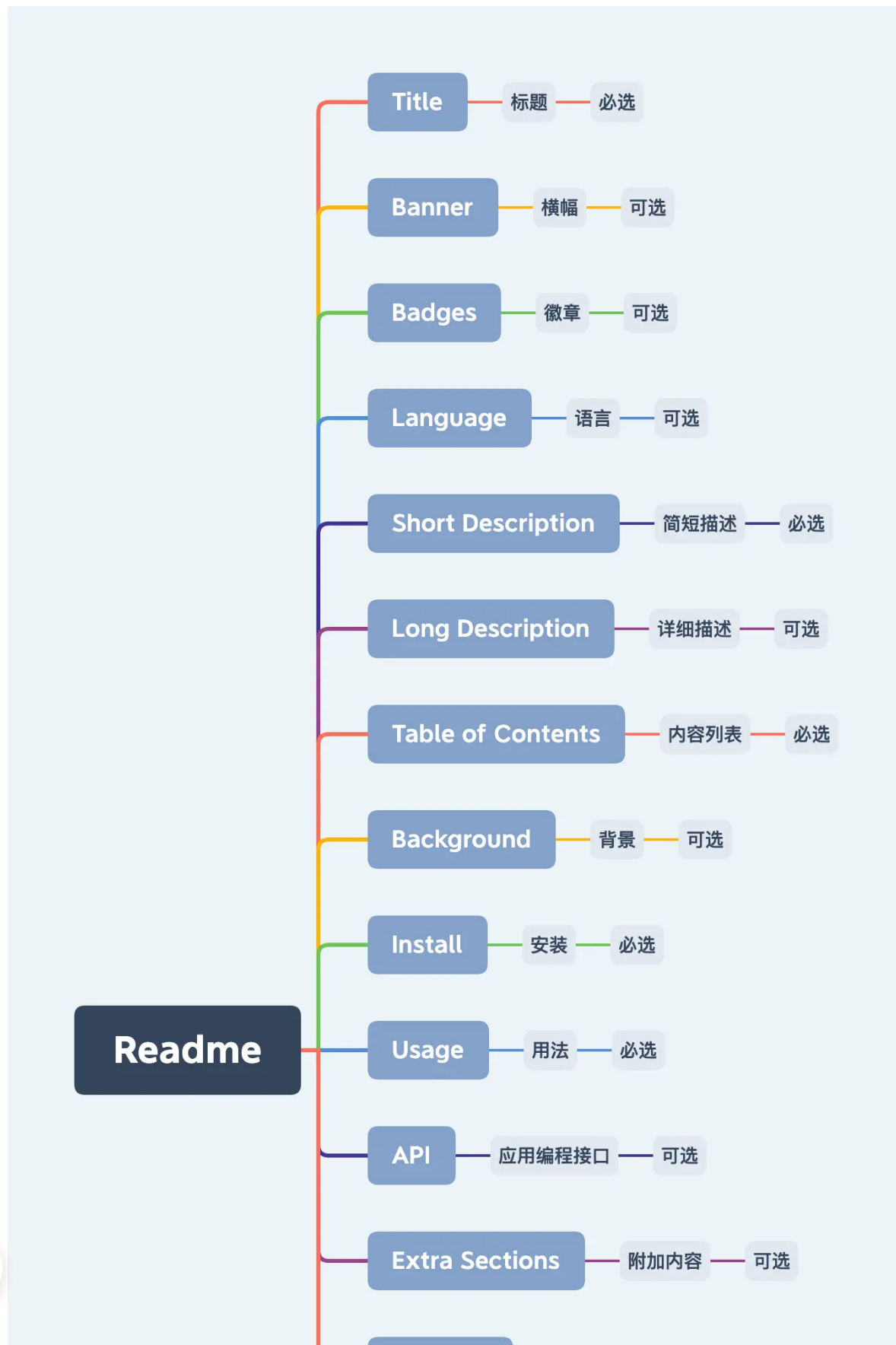
方案：优化项目自述文件

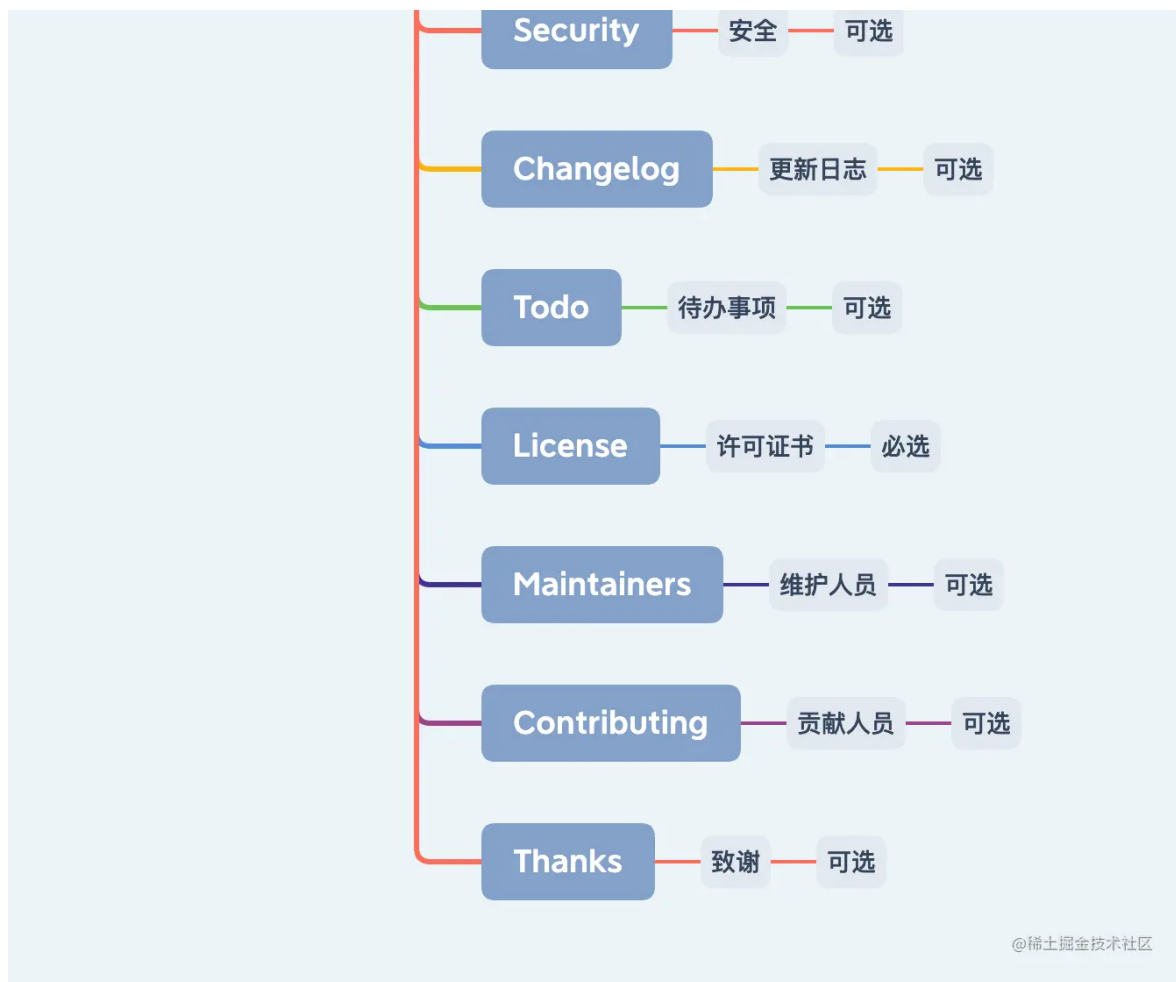
Readme指项目自述文件，用于为用户提供项目详细信息。与产品说明书很像，包括但不限于背景、安装、使用、徽章、示例、维护、贡献、证书等信息。 [Readme](#) 对应实体文件是 `readme.md` 或 `README.md` 。

很多项目越来越重视写好一个 [Readme](#) ，因为 [Readme](#) 最能直接反映项目的质量与好坏。优秀项目不一定有一个好的 [Readme](#) ，但不好的 [Readme](#) 一定不是一个优秀项目。

标准规范

Github 中有一个堪称教科书式的 **Readme** 项目 [standard-readme](#)，它为开发者提供一份标准化的编写规范。为了方便编写 **Readme**，我将其翻译下，得出一篇完善的 **Readme** 需由以下部分组成，包括必选与可选。



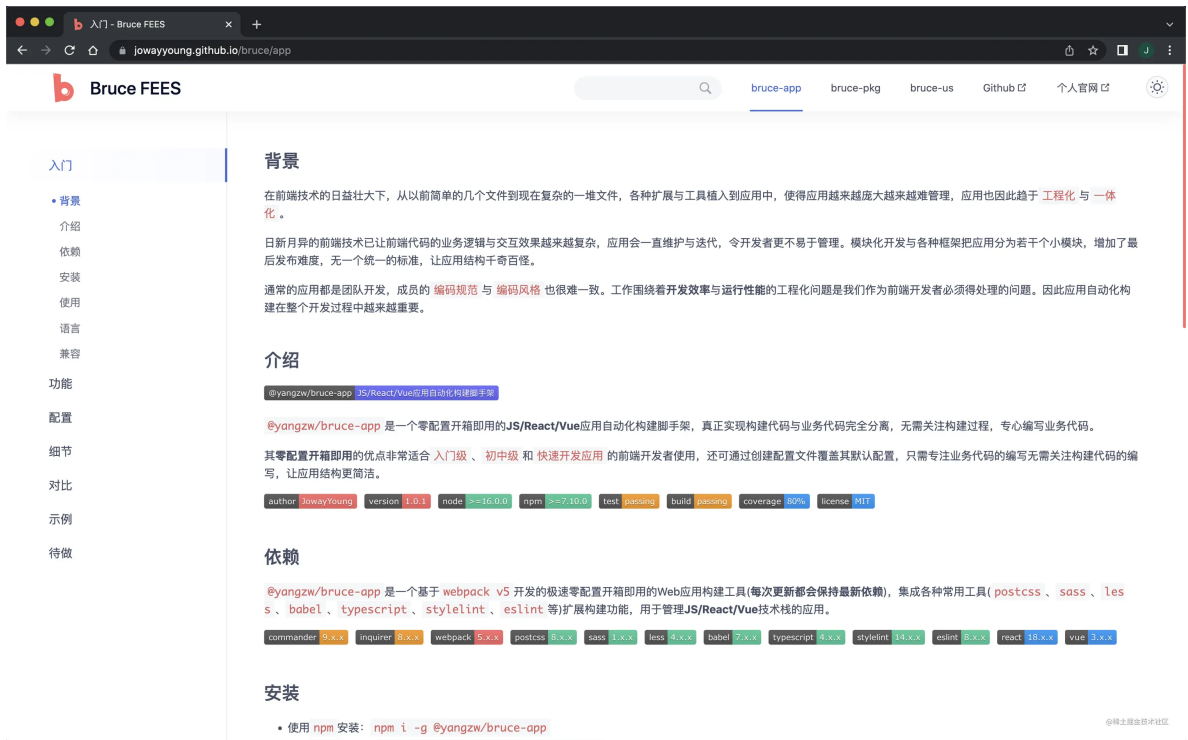


基本上根据上述组成部分就能编写一份标准规范的 **Readme** 了，一份最基本的 **Readme** 必须由以下部分组成。

- ✓ **Title**：标题
- ✓ **Short Description**：简短描述
- ✓ **Table of Contents**：内容列表
- ✓ **Install**：安装
- ✓ **Usage**：用法
- ✓ **License**：许可证书

徽章图标

平时逛 **Npm** 与 **Github** 会发现很多仓库的 **Readme** 都会贴上五颜六色的徽章图标，这些图标美观好看又方便，为清一色的 **Readme** 加上了点睛之笔。我的开源项目 [bruce](#) 的 [文档站点](#) 也使用了很多美观好看的徽章图标点缀文档内容。



其实这些徽章图标都是由[Shields](#)动态生成。[Shields](#) 是一个以 [SVG](#) 或光栅格式提供简洁清晰的徽章服务，可轻松地应用在 [Readme](#) 或任何网页中。

该服务支持数十种持续集成服务、软件包注册表、发行版、应用商店、社交网络、代码覆盖率服务和代码分析服务，每个月提供超过 [7.7亿](#) 张图像。基本上每个受欢迎的开源项目都会使用 [Shields](#) 提供的徽章服务。

手动生成

打开[Shields](#)，填写 [label](#)标签 与 [message](#)消息，选择 [color](#)颜色，点击 [Make Badge](#) 就会生成一个 [URL](#)，该 [URL](#) 其实就是一个动态生成的图像。

`img.shields.io/badge/BMW-330Li-blue` md

仔细观察 [URL](#) 与生成图像，图标由左右两部分组成，左边是 [label](#)标签，右边是 [message](#)消息 与 [color](#)颜色，由此可推算出徽章图标的生成公式是 [https://img.shields.io/badge/\[label\]-\[message\]-\[color\]](#)。



自动生成

有了上述公式，可根据自己想法随意拼接出不同徽章图标了。颜色除了可用 颜色关键字，还可用 HEX、RGB、RGBA、HSL 和 HSLA 表示，若使用 HEX 表示，无需加上 #。

```
img.shields.io/badge/Name-JowayYoung-f66
img.shields.io/badge/Sex-Male-66f
img.shields.io/badge/Age-29-f90
img.shields.io/badge/Area-Guangzhou-09f
```

md



除了这种常见 URL 生成方式，还可在 URL 后方加上参数实现定制。

logo与logoWidth

通过 logo 设置左边内容的图标，这些图标可通过[SimpleIcons](#)获取。

```
img.shields.io/badge/@yangzw/bruce--app-1.0.0-f66?logo=npm
```

md



通过 logoWidth 设置左边内容的图标宽度。该宽度并不是图标的宽度，而是图标的外层宽度。

```
img.shields.io/badge/@yangzw/bruce--app-1.0.0-f66?logo=npm&logoWidth=50
```

md

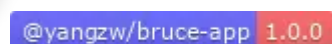


labelColor

通过 labelColor 设置左边内容的背景色。背景色默认是 #5d5d5d，可用 颜色关键字、HEX、RGB、RGBA、HSL 和 HSLA 表示，若使用 HEX 表示，无需加上 #。

```
img.shields.io/badge/@yangzw/bruce--app-1.0.0-f66?labelColor=66f
```

md



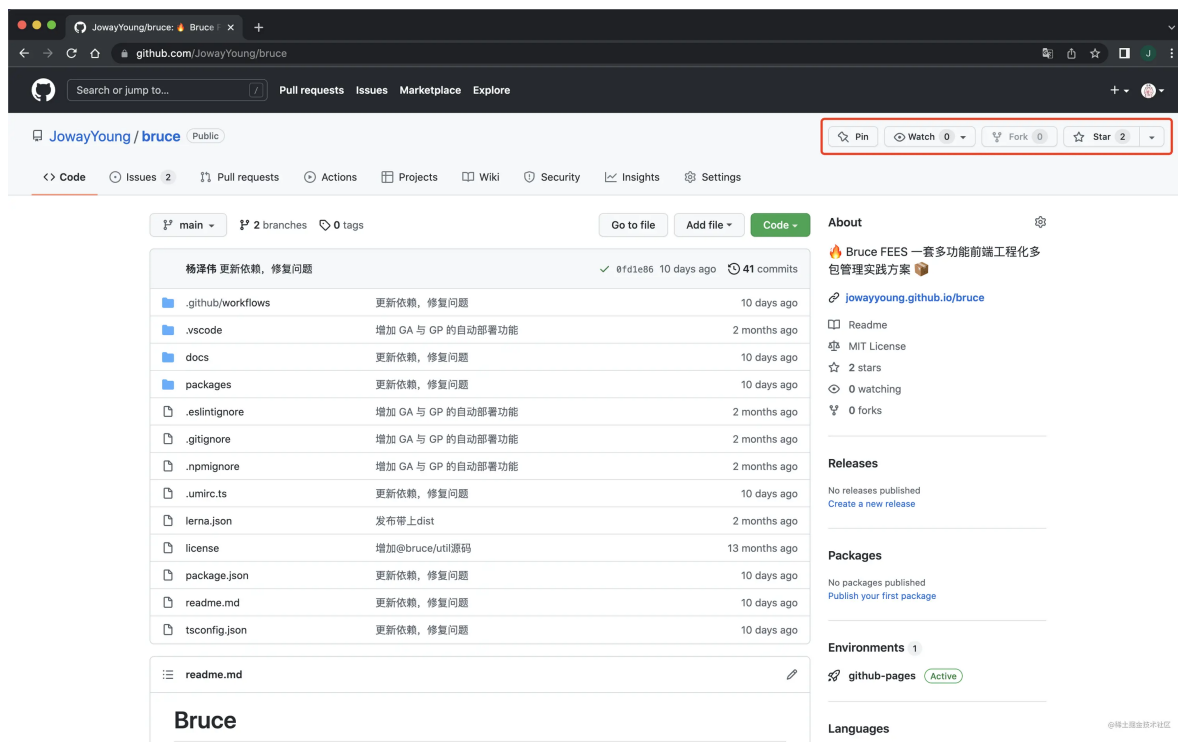
link

通过 `link` 设置左右两边内容的链接。第一个 `link` 表示左边内容的链接，第二个 `link` 表示右边内容的链接。

```
img.shields.io/badge/@yangzw/bruce---app-1.0.0-f66?link=https://yangzw.vip&link=https://jmd
```

@yangzw/bruce-app 1.0.0

另外 `Github` 也提供类似的徽章服务，若要实时展示自己项目的 `Star`、`Fork` 和 `Watch`，可在 `Readme` 中加入以下内容。



```
<iframe src="https://ghbtns.com/github-btn.html?user=JowayYoung&repo=bruce&type=star&size=md
<iframe src="https://ghbtns.com/github-btn.html?user=JowayYoung&repo=bruce&type=fork&size=
<iframe src="https://ghbtns.com/github-btn.html?user=JowayYoung&repo=bruce&type=watch&size=
```

Star 2 Fork 0 Star 2

@稀土掘金技术社区

从三条 URL 可知，整体链接为 <https://ghbtns.com/github-btn.html>，通过加上以下参数动态生成对应统计数据，然后以 `<iframe>` 加入到 Readme。因为 md 文件 本身就是一个特殊的 html 文件，所以加入 `<iframe>` 也是能正常渲染的，除非 MD 引擎 自动把 `<iframe>` 过滤掉。

- **user**: 用户名称
- **repo**: 仓库名称
- **type**: 统计类型，可选 `star/fork/watch`
- **size**: 图标尺寸，可选 `small/larget`
- **count**: 是否显示计数，可选 `true/false`

因为是实时渲染，相信不少开发者关注 [bruce](#) 后，每次看都会看到不一样的统计数据。

总结

本章更多是理论与实践相结合的内容，所以必须自己动手操作一次。通过从 [项目配置文件](#)、[软件许可证书](#) 和 [项目自述文档](#) 三方面入手处理好部署项目前的细节问题，也是一件让项目变得更好的事情，毕竟细节决定成败，我也相信任何细节都能导向好的结果。趁热打铁，赶紧把自己项目的细节问题处理好吧！

本章内容到此为止，希望能对你有所启发，欢迎你把自己的学习心得打到评论区！

- ☒ 示例项目: [fe-engineering](#)
- ☒ 正式项目: [bruce](#)

留言

输入评论 (Enter换行, Ctrl + Enter发送)

发表评论

